
When Does Monitoring Long-Horizon Agents Lead to Better Performance?

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Long-horizon language-model agents often rely on runtime monitors to detect
2 degradation and trigger interventions in their workflows. We study whether such
3 monitoring actually improves agent performance. Across behavioral sentinels,
4 multiple models, and long-horizon tasks, we find that monitoring is not a free
5 observation: there exists an “observer effect” where probes can alter the behavior
6 of the agent they are meant to measure and that no single monitoring method dom-
7 inates across all settings. More importantly, we find that the value of monitoring
8 depends strongly on the exact recovery mechanism it triggers. Our results show
9 that agent monitors should be evaluated jointly with the recovery systems they are
10 paired with, rather than as standalone predictors.

11 1 Introduction

12 The use of agents like Claude Code and Codex has been drastically increasing for tasks that
13 require maintaining goals and state over an extended period of time(i.e deep research, coding,
14 etc).[CITATION] As these trajectories grow longer, however, agents can gradually lose important
15 context and start hallucinating incorrect information[CITATION]. A natural response is to monitor
16 an agent during execution and intervene whenever its behavior starts to degrade. This can include
17 restarting the agent compacting its context or simply restructuring state before the agent continues
18 execution[CITATION(S)].

19 The need to preserve agent behavior creates an appealing intuition: if we can detect degradation
20 earlier and more accurately, we should be able to intervene at better times and improve task perfor-
21 mance. This has drastic implications for all agent tasks, whether it be coding benchmarks like (...) or
22 enterprise (...).

23 Much of agent monitoring can thus be viewed as a prediction problem: given an agent current
24 behavior/state, determine whether it is likely to fail in the near future. Under this view, a more
25 accurate monitor should naturally produce a better agent.

26 *[insert a conditional prob math thing here explaining the above notion, or can put in experimental*
27 *setup]*

28 We argue this view is incomplete; any runtime monitor is not necessarily a passive observer of an
29 agent trajectory.

30 The most common form of monitoring introduces additional instructions or computations directly
31 into the agent context[CITATION], [maybe include a quick diagram example here as well]. We study
32 one such class of monitors, which we call behavioral sentinels: canaries that an agent must maintain
33 throughout a task and whose degradation can serve as a warning signal. For example, an agent might
34 be asked to remember and update a small piece of state while simultaneously completing its primary

task. The intuition here is if the agent fails to maintain the state before completing the underlying task, the sentinel can provide an early warning signal.

However, asking the agent to maintain such a signal can itself change the trajectory of an agent. Across our experiments, we find an observer effect: a probe cannot predict the degradation of an agent without playing some part in the degradation itself. Monitoring therefore is not always a free source of information, and higher quality signals do not automatically imply higher performance.

Detecting degradations is useful because the resulting signal eventually triggers an intervention. Intuitively, this means the value of a monitor may depend not just when it fires but what the system does afterwards. We study this interaction directly by evaluating monitoring under different recovery mechanisms. Specifically, to answer the question of whether the effect of monitoring depends on the recovery mechanism it is paired with, we compare recovery based on agent-generated context compaction with recovery that reconstructs state from an external source of truth.

Our experiments reveal that it does. Under lossy-agent generated compaction, carrying additional state can reduce the benefit of an otherwise well-timed intervention. However, when the state is reconstructed through deterministic regrounding, this penalty disappears and the monitoring setup can be beneficial.

Our work makes three primary contributions:

- We characterize monitoring as an intervention rather than a free observation. Behavioral probes alter trajectories, producing an observer effect
- We experimentally isolate the interaction between monitoring and recovery. Holding the monitoring setup fixed, we show that changing how agent state is reconstructed can substantially change whether monitoring improves downstream performance.
- We evaluate monitoring as a complete systems problem. Across multiple models and long-horizon tasks, we find that no single monitoring strategy consistently dominates.

2 Related Works

2.1 What has already been done

Recent work has explored detecting degradation on agents before task completion

[Citation-> Doomed from the start] introduces the idea of utilizing probes over internal model activations to predict task failure and terminate struggling trajectories. [Citation] similarly detects agent drift from internal activation and uses activation steering to mitigate it during execution.

Other work has introduced explicit diagnostic probes: [Canary Tools], for example, inserts adversarially designed tools into an agent’s context to expose weaknesses in tool-selection reasoning.

Together, these methods show us that agent failure can often be detected before task completion, whether it be internal representations, observing trajectories, or introducing probes

[Show diagram here of the three methods and agent]

2.2 What gap remains

A newer line of work explores what should happen after degradation is detected. [Citation] combines early failure prediction with different restart strategies, showing us that preserving useful work across sessions can outperform a cold restart. [AgentRewind] further treats recovery as a state management problem, checkpointing both agent context and environment state, so execution can return to an earlier point after agent failure.

Separately work on long-horizon context management has shown that LLM-generated compaction is inherently lossy and can unpredictably discard important information that is needed for later sessions[CITATION].

79 2.3 Why this paper is different

80 Prior work therefore establishes two notions: agent failures can be monitored and that choosing
81 the right recovery strategy matters. However, these components are typically evaluated separately.
82 Monitoring work primarily asks whether a signal predicts a failure, while recovery work asks how
83 best to restore or restart an agent once intervention occurs. To our knowledge, prior work has not
84 studied how the monitoring mechanism itself interacts with agent state after intervention. We study
85 this precise interaction explicitly. Using behavioral sentinels and passive monitors, we measure both
86 signal quality and how observation changes task execution and recovery.

87 In particular, we hold the monitoring setup fixed while replacing lossy agent-generated compaction
88 with deterministic external re-grounding, allowing us to precisely test whether the effect of monitor-
89 ing changed depending on the recovery mechanism.

90 We need to “get-to-the-point” faster into our conclusions below

91 3 Experiment Set up

92 A little on experiment set up

93 3.1 Agentic Tasks — NO regard for observation methods

94 incremental resets vs no resets — how does performance differ?

95 incremental regrounding vs no regrounding — how does performance differ?

96 3.2 Error Recall — JUST active observation

97 Active probes vs baselines — leads to pretty bad the precision / recall

98 This is because of “observer effect”, where the extra chore confounds performance on the task itself

99 3.3 Findings on when it’s best to compact / re-retrieve context stored in agent state

100 4 Results

101 5 Discussion / Conclusion

Recovery changes performance—and depends on the monitor

Natural-policy deployment: operator effect relative to monitored no-action

GPT-5.6 Luna × Evolving Intent; n = 40 paired source tasks; bars are paired-bootstrap 95% CIs

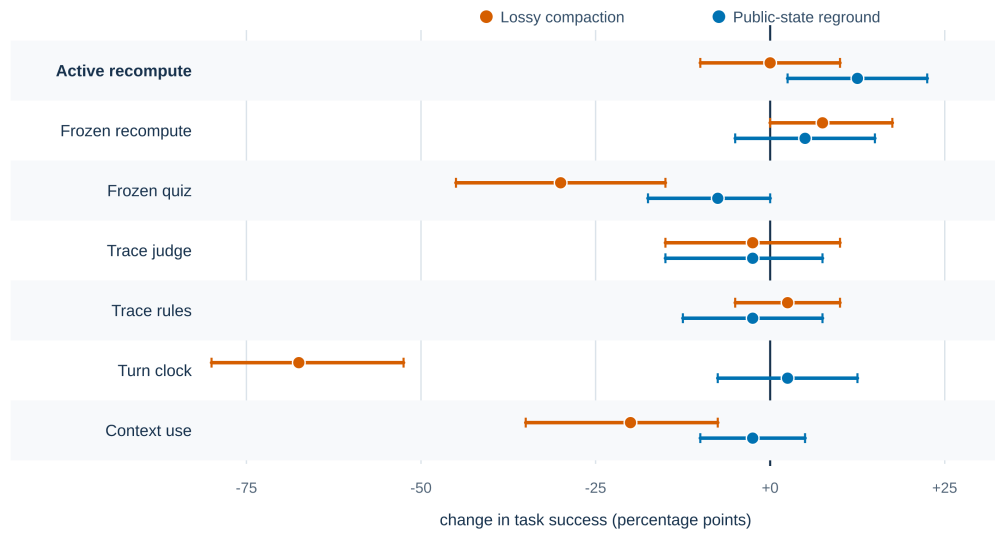


Figure 1: Recovery-operator effects relative to monitored no-action in the natural-policy deployment. Lossy compaction and deterministic public-state regrounding produce different outcomes, and their effects vary with the paired monitoring method.

Observer effect of carried active probes

Paired change in task success (active – clean); negative values mean the carried probe harmed success

■ harm ■ benefit □ 95% paired-bootstrap CI excludes zero

A · Exploratory mechanism screen — n = 20 paired tasks per cell

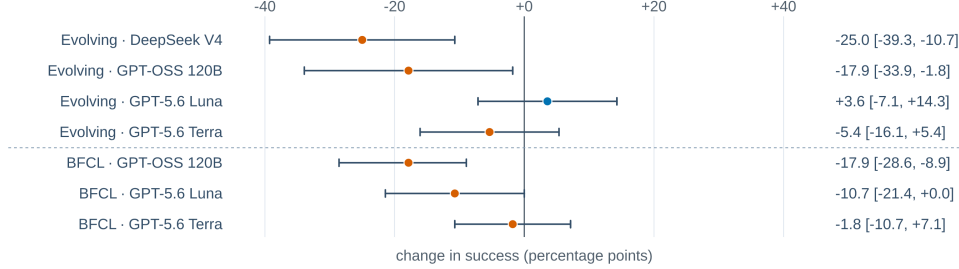
Display order is conceptual burden (copy → memory → update → reasoning), not a validated dose; output length is fixed.

	Copy current code	Recall initial code	Update carried counter	Recompute arithmetic	pattern
Evolving · DeepSeek V4	-20	-50	-65	+0	mixed
Evolving · GPT-OSS 120B	-10	-10	-30	-10	mixed
Evolving · Qwen3P7+	+0	+10	-5	+5	mixed
Evolving · GPT-5.6 Luna	+25	+25	+15	+25	mixed
Evolving · GPT-5.6 Terra	+10	+5	+5	-10	worsens
BFCL · GPT-OSS 120B	-10	-5	-15	-15	mixed
BFCL · Qwen3P7+	+0	+0	+0	+0	flat
BFCL · GPT-5.6 Luna	-5	-10	+0	+15	mixed
BFCL · GPT-5.6 Terra	-10	-5	-5	+0	opposite

Cells: percentage-point effects. "Worsens" = non-increasing; "opposite" = non-decreasing; "mixed" = changes direction.

B · Powered, prespecified recompute contrast — n = 56 paired tasks per cell

Separate tasks and inference tier. Points: active – clean; lines: 95% paired task-bootstrap intervals.



Descriptive pattern, not a pooled effect

Exploratory: 18 negative / 8 zero / 10 positive; only 1 of 9 non-flat strata shows monotone worsening.

Adjacent exploratory steps: 9 worsen / 10 improve / 8 tie. Powered recompute: 6/7 negative; 3/7 CIs strictly below zero.

Conclusion: active-probe harm is a cross-model trend for recompute, not a universal rule or a simple complexity dose-response.

Cells are model × benchmark contrasts. Tasks repeat across models; sign counts are descriptive, not independent observations.

Figure 2: Observer effect of carried active probes. The lower panel is the powered, prespecified recomputation contrast; the upper mechanism screen is exploratory.

No monitoring method dominates across settings

Area under the precision–recall curve (AUPRC); darker cells indicate better signal quality

Outlined cells are the best method within each model × task slice



Figure 3: Signal quality across every powered model-by-task slice. The identity of the best method varies across settings, so neither active, passive, nor baseline monitoring dominates universally.

Observation itself consumes compute

Total observer tokens across 392 model–task cases; annotations show recorded observer cost

Provider-backed methods each made 1,764 calls; deterministic rules, clock, and context-use made none

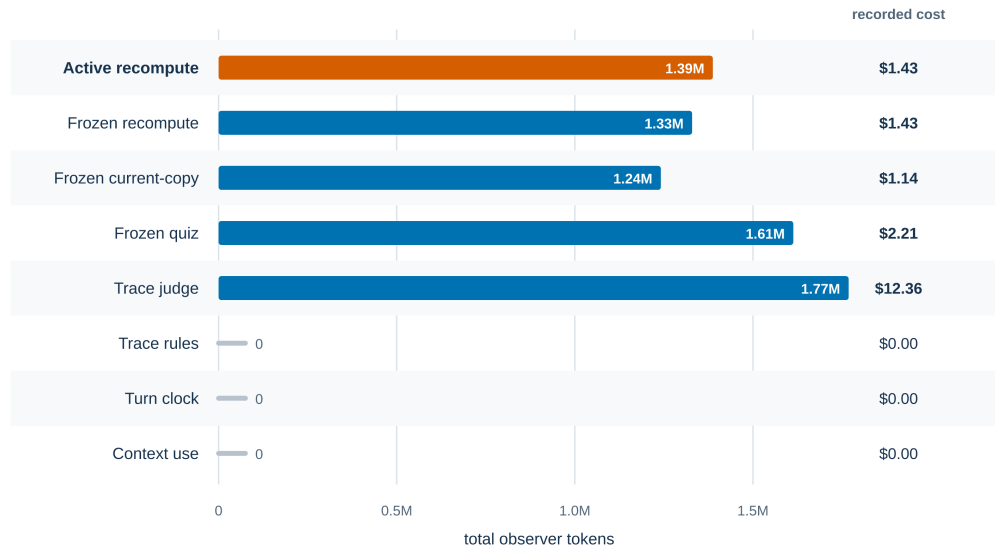


Figure 4: Compute used by observation itself. Provider-backed methods incur calls, tokens, and recorded cost; deterministic trace rules, clocks, and context-use add no provider calls.

